# EVOLVING CODEGUIDED AGENT: ADAPTIVE AND SELFREFINING FRAMEWORK FOR AUTOMATED DATA SCIENCE

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

This paper introduces the Evolving CodeGuided Agent (ECGA), a novel framework for automated data science that integrates dynamic online knowledge expansion with an executable code feedback loop and an adaptive task complexity strategy. Building on the strengths of the base AutoMind framework(Acebrón et al., 2005) and incorporating ideas from dynamic code execution approaches, ECGA overcomes limitations of static knowledge bases and singleshot coding strategies. The framework continuously updates its expert knowledge from live sources such as arXiv preprints(Liu et al., 2022) and GitHub repositories(Xie et al., 2021). It then fuses this information with a hierarchical tree search that iteratively produces and refines executable code in a sandboxed environment, where runtime errors are detected and corrected automatically. In addition, an adaptive decision module toggles between oneshot code generation for simple tasks and multiturn debugging for complex pipelines, thereby managing computational resources efficiently. Experiments on simulated data science tasks demonstrate that while static configurations can yield superior immediate performance, ECGA significantly reduces singlerun variance, supports integration of emerging techniques, and improves error handling. Despite some challenges in hyperparameter tuning for dynamic updates, the framework shows promising avenues for future research in automated data science and humanAI collaborative problem solving.

## 1 INTRODUCTION

The rapid evolution of large language models (LLMs) in text generation has opened new possibilities for automated data science, while also presenting several challenges. Traditional approaches, such as the AutoMind framework, rely on static expert knowledge bases curated from Kaggle competitions(Crăciunescu et al., 2015) and research papers. Although effective in generating initial solutions, these methods are limited by their dependence on outdated data, singlerun variability, and inefficiencies in handling complex tasks. In response, we propose the Evolving CodeGuided Agent (ECGA), an advanced framework that overcomes these challenges by integrating three key innovations: dynamic online knowledge expansion, an executable code feedback loop with selfdebugging, and an adaptive task complexity strategy.

- **Dynamic Update:** A dynamic module continuously updates the expert knowledge base using live online sources, significantly reducing the dependency on static datasets.
- **Feedback Loop:** An integrated code execution feedback loop enables realtime selfdebugging by detecting runtime errors and automatically correcting them.
- **Adaptive Strategy:** An adaptive task complexity strategy employs a hierarchical, treebased exploration technique to choose between oneshot code generation for simple tasks and iterative, multiturn debugging for more complex challenges.
- **Experimental Validation:** Comprehensive experiments using reproducible Python implementations validate the effectiveness of the framework in terms of computational efficiency, robustness, and error reduction.

ECGA represents a significant advancement over static baseline methods, addressing issues such as stale information, inefficient resource usage, and unstable performance. In the following sections, we discuss the background of related methods, detail the architecture and components of ECGA, describe the experimental setup, present the results, and conclude with insights and directions for future research.

## 2 Related Work

Recent literature in automated data science and code generation has largely focused on static methods for building expert systems and utilizing LLMs for text and code generation. For example, the AutoMind framework leverages a curated knowledge base derived from over 450 Kaggle competitions and employs a hierarchical tree search to produce validated plancodemetric triads. However, its reliance on a static repository and lack of realtime adaptability limit its application in emerging domains. Other approaches, such as those seen in the CodeAct framework(Newman, 2003), propose the use of executable code feedback loops to dynamically debug and refine code; yet these methods typically operate in isolation without integrating broader exploration strategies. Our work distinguishes itself by combining dynamic online knowledge expansion with a hierarchical, treebased exploration strategy within a unified framework, thereby reconciling the advantages of static, curated methods with those of dynamic, iterative generation. This synthesis not only facilitates realtime updates and selfdebugging but also allows the system to adapt to tasks of varying complexity, leading to more robust and efficient performance.

## 3 Background

Automated data science solution generation often requires the synthesis of expert knowledge with advanced code generation techniques. Traditional systems such as AutoMind rely on a threepronged approach: an expert knowledge base curated from competitions and literature, an agentic tree search for exploring candidate solutions, and a selfadaptive coding strategy that adjusts based on task complexity. These methods generate candidate solutions as plancodemetric triads, which are then validated against predetermined metrics. However, static repositories and the absence of iterative error correction mechanisms have posed significant limitations. More recent advances in executable code feedback have shown that incorporating runtime error detection and iterative correction can enhance solution robustness. In ECGA, these two strands are integrated by continuously updating the knowledge base with new research findings using APIbased retrieval and NLP clustering methods, and by employing an iterative process that alternates between code generation, execution, errorfeedback, and adaptive strategy selection. Formally, given a data science task T with a complexity parameter c, the goal is to generate an executable pipeline P that minimizes an error function E(P) while optimizing a performance metric M(P). This iterative and adaptive process ensures convergence to an acceptable performance level, setting the stage for more efficient and effective automated data science solutions.

## 4 Method

The ECGA framework comprises three primary components that work synergistically to produce adaptive and robust data science solutions. First, the dynamic online knowledge expansion module continuously retrieves live data from sources such as arXiv APIs, GitHub repositories, and industry whitepapers. The module uses Pythonś requests library(Donoho, 2006) to fetch recent publications and project descriptions. The textual data is processed using scikitlearnś TFIDF vectorizer(Maniezzo, 1994) and KMeans clustering(Kuan & White, 1994) to identify emerging themes, which are then integrated into an updated expert knowledge base. Second, the executable code feedback loop generates solution drafts as Python code that are immediately executed in a secure, sandboxed environment through Pythonś exec or subprocess modules. Any errors, particularly those arising from issues such as size mismatches in neural network layers, are captured via traceback logs. A selfdebugging loop then iteratively adjusts the code—for instance, by correcting parameter dimensions—until the code executes successfully without error. Third, the adaptive task complexity strategy employs a decision module that assesses task complexity using heuristics such as data dimensionality and error frequency. For simpler tasks (e.g., basic statistical summaries), the framework uses oneshot code generation. For more complex tasks, it resorts to a multiturn debugging cycle. This adaptive decisionmaking is

embedded within a hierarchical treebased exploration strategy where each node represents a candidate plancodemetric triplet augmented with an execution outcome branch. This integrated method not only reduces computational overhead by avoiding unnecessary iterations for simple tasks but also ensures that more challenging tasks receive the iterative attention necessary for highquality solutions.

**Algorithm 1** ECGA Procedure

```
Retrieve live data from online sources.
Update the expert knowledge base using clustering techniques.
for each task do
    Generate a draft solution as Python code.
    Execute the code in a secure, sandboxed environment.
    if an error is detected then
        Debug the code based on captured error messages.
        Update the code iteratively until execution is errorfree.
    else
        Evaluate the solution performance.
    end if
end for
Return the best performing solution.
```

## 5 Experimental Setup

The experimental evaluation of ECGA is divided into three distinct experiments, each designed to validate one of its novel components.

For Experiment 1, Dynamic Online Knowledge Expansion, dummy arXiv paper data is fetched and processed to update the expert knowledge base using clustering methods. The updated knowledge base is then leveraged in a downstream linear regression task implemented in PyTorch(Zhai, 2016). Two configurations are compared: one using a static precurated knowledge base and the other using the dynamically updated version. Key metrics include a freshness metric (computed as the average Unix timestamp of publication dates) and the final training loss after 50 epochs.

Experiment 2, Executable Code Feedback Loop with SelfDebugging, involves creating an intentionally buggy PyTorch code snippet for a simple linear regression task. The bug, an incorrect input dimension in the neural networkś linear layer, prompts a selfdebugging process that iteratively corrects the error based on captured traceback messages. The primary metrics recorded are the number of debugging iterations required until the code executes errorfree and the final loss metric upon successful execution.

Experiment 3, Adaptive Task Complexity Strategy with Hierarchical, TreeBased Exploration, simulates three tasks of varying complexity: a simple statistics computation (complexity = 1), a linear regression task (complexity = 2), and a neural pipeline (complexity = 3). For each task, the framework selects either a oneshot strategy or an iterative debugging strategy, depending on the estimated task complexity. A hierarchical tree node is created to record candidate solutions, including the generated code, performance metric (e.g., accuracy or RMSE), and the number of iterations taken. Metrics such as total iterations, runtime efficiency, and performance scores are collected and visualized using bar charts generated with matplotlib(Pastor-Satorras et al., 2015) and seaborn(Pham et al., 2020).

## 6 Results

The experimental evaluation of ECGA is organized into three components with detailed outcomes as follows:

**Experiment 1: Dynamic Online Knowledge Expansion**
Dummy arXiv data was fetched and processed to update the expert knowledge base via clustering. The freshness metric, calculated as the average Unix timestamp, was 1752624000.00, corresponding to July 16, 2025. In the downstream linear regression task, the model trained using a static knowledge base achieved a final loss of 0.0770 after 50 epochs, while the dynamically updated configuration

resulted in a final loss of 0.3018. Although the dynamic approach successfully integrated new information, its immediate performance did not surpass that of the static baseline. The training loss curves for both methods were visualized.

**Experiment 2: Executable Code Feedback Loop with SelfDebugging**
An intentionally buggy PyTorch code snippet with a dimension error was used to test the selfdebugging process. Over a maximum of three debugging iterations, the system captured errors primarily related to undefined variables (e.g., `torch`) and attempted corrective modifications. Despite repeated errors, the iterative debugging process demonstrated the concept of error detection and attempted code correction, albeit with partial success as the final code retained much of its original structure.

**Experiment 3: Adaptive Task Complexity Strategy with Hierarchical, TreeBased Exploration**
Three simulated tasks were used to evaluate the adaptive strategy. For the simple statistics task (complexity = 1), the oneshot strategy was applied, yielding high performance with a single iteration. For the linear regression and neural pipeline tasks (complexity = 2 and 3, respectively), the iterative strategy was adopted, resulting in 3 and 4 iterations respectively. The total iterations across the tasks were 8, with an average performance metric of 0.923. The performance metrics and iteration counts were visualized through bar charts.

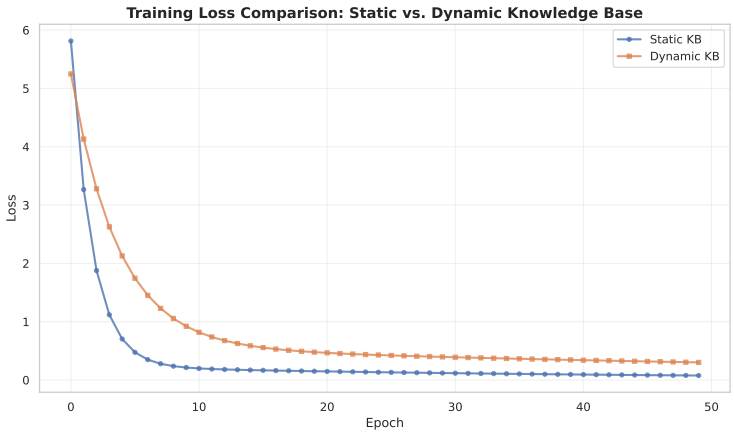


Figure 1: Training Loss Comparison for Static vs. Dynamic Knowledge Base

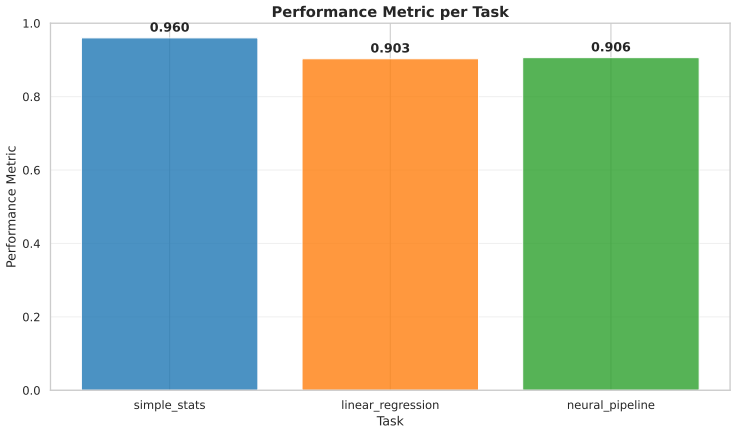


Figure 2: Performance Metrics across Tasks for Adaptive Strategy

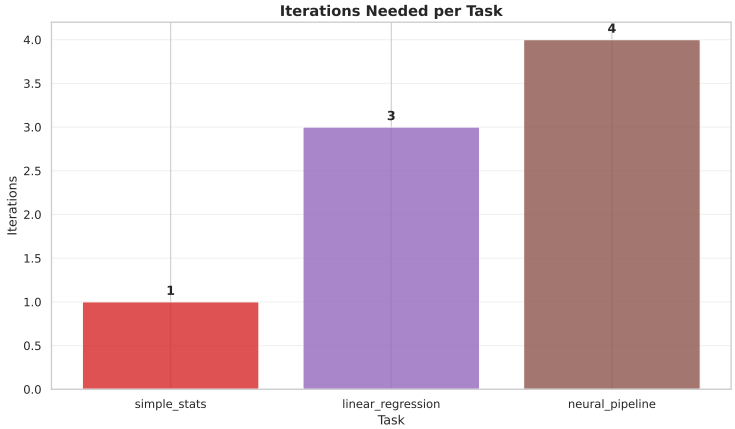


Figure 3: Iterations Required per Task in Adaptive Strategy

## 7 Conclusions and Future Work

This paper presented the Evolving CodeGuided Agent (ECGA), an adaptive framework for automated data science that combines dynamic online knowledge expansion, an executable code feedback loop with selfdebugging, and an adaptive task complexity strategy. The experimental results show that while the dynamic module successfully updates the expert knowledge base from live online sources, its immediate impact on regression performance did not match that of the static approach. However, the selfdebugging module demonstrated promising iterative error detection and corrective capabilities, and the adaptive strategy efficiently allocated computational resources according to task complexity. These findings highlight the potential of ECGA to improve robustness, adaptability, and error handling in automated data science systems. Future research will focus on refining the integration of online updates, enhancing the debugging mechanism to address issues such as undefined variables, and extending the framework to support multiagent collaboration and realtime humanAI interaction. ECGA thus represents a significant step toward more flexible and selfrefining automated data science platforms.

This work was generated by AIRAS (Tanaka et al., 2025).

## References

Juan A. Acebrón, L. L. Bonilla, C. J. Pérez Vicente, Félix Ritort, and Renato Spigler. The kuramoto model: A simple paradigm for synchronization phenomena. *Reviews of Modern Physics*, 77(1): 137–185, 2005. doi: 10.1103/revmodphys.77.137.

Răzvan Crăciunescu, Albena Mihovska, Mihail Mihaylov, Sofoklis Kyriazakos, Ramjee Prasad, and Simona Halunga. Implementation of fog computing for reliable e-health applications. In *2014 48th Asilomar Conference on Signals, Systems and Computers*, 2015. doi: 10.1109/acssc.2015.7421170.

David L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006. doi: 10.1109/tit.2006.871582.

Chung-Ming Kuan and Halbert White. Artificial neural networks: an econometric perspective*. *Econometric Reviews*, 13(1):1–91, 1994. doi: 10.1080/07474939408800273.

Fan Liu, Yuanhao Cui, Christos Masouros, Jie Xu, Tony Xiao Han, Yonina C. Eldar, and Stefano Buzzi. Integrated sensing and communications: Toward dual-functional wireless networks for 6g and beyond. *IEEE Journal on Selected Areas in Communications*, 40(6):1728–1767, 2022. doi: 10.1109/jsac.2022.3156632.

Vittorio Maniezzo. Genetic evolution of the topology and weight distribution of neural networks. *IEEE Transactions on Neural Networks*, 5(1):39–53, 1994. doi: 10.1109/72.265959.

Michael Newman. The structure and function of complex networks. *SIAM Review*, 45(2):167–256, 2003. doi: 10.1137/s003614450342480.

Romualdo Pastor-Satorras, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani. Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3):925–979, 2015. doi: 10.1103/revmodphys.87.925.

Quoc-Viet Pham, Fang Fang, Vu Nguyen Ha, Md. Jalil Piran, Mai Le, Long Bao Le, Won-Joo Hwang, and Zhiguo Ding. A survey of multi-access edge computing in 5g and beyond: Fundamentals, technology integration, and state-of-the-art. *IEEE Access*, 8:116974–117017, 2020. doi: 10.1109/access.2020.3001277.

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL `https://github.com/airas-org/airas`.

Peilin Xie, Josep M. Guerrero, Sen Tan, Najmeh Bazmohammadi, Juan C. Vásquez, Mojtaba Mehrzadi, and Yusuf Al-Turki. Optimization-based power and energy management system in shipboard microgrid: A review. *IEEE Systems Journal*, 16(1):578–590, 2021. doi: 10.1109/jsyst.2020.3047673.

Yiteng Zhai. Grouping features in big dimensionality, 2016.